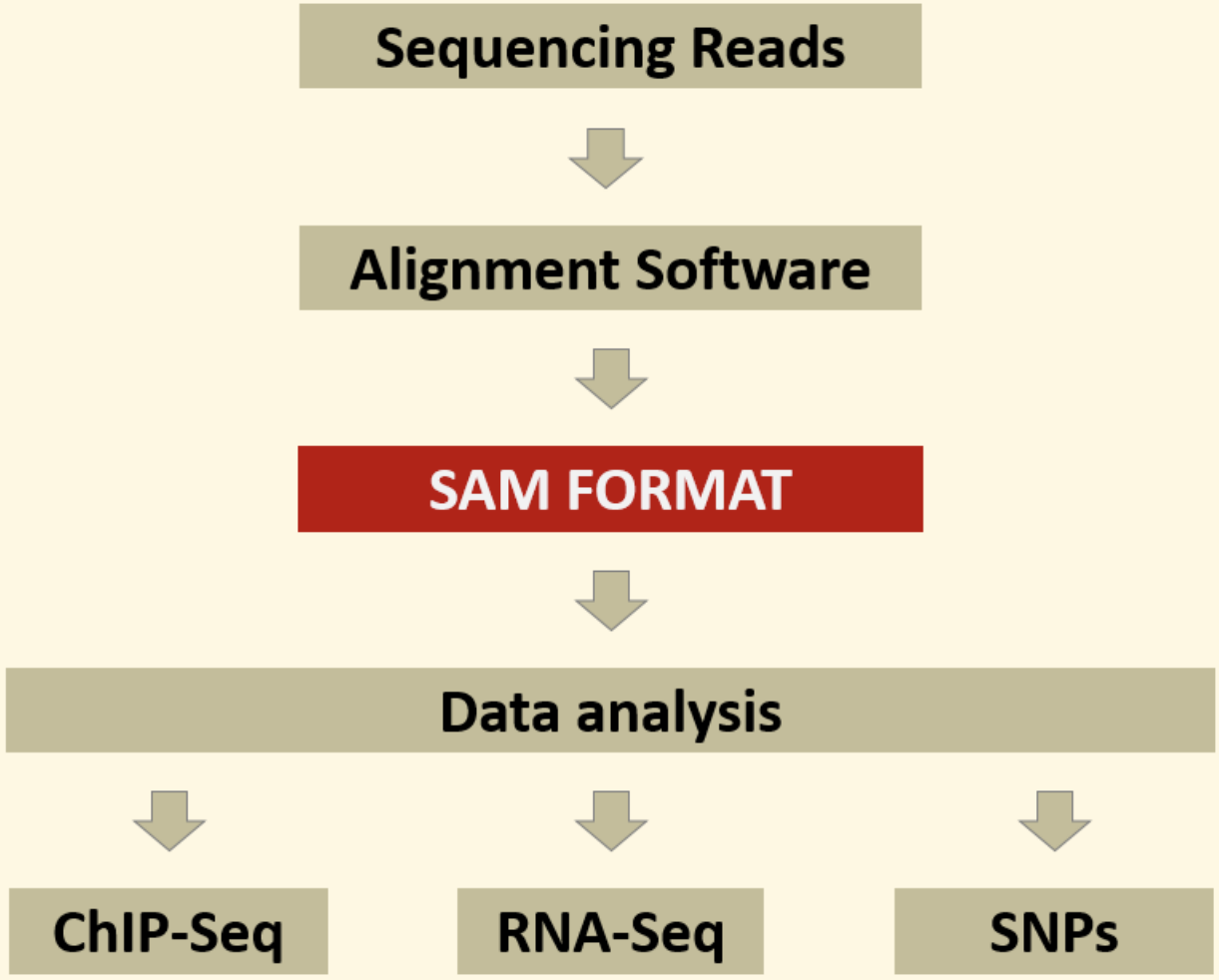
Sequence Alignment Maps

The SAM Format

Sequence Alignment Maps: SAM



What is a SAM file

The "standard" result of a short read aligner.

Understand your SAM file --> **Understand** your data.

The promise of SAM was to define a format that contains all information a scientists might ever need.

A big promise - no wonder it can't quite live up to it.

Understanding SAM files essential

The majority of analyises work off **SAM files**.

Getting the **correct SAM** file and understanding what does and what it does not contain is essential.

Performing a bioinformatics analysis usually means **mining**, **exploring**, **summarizing** the SAM files.

Not all is good with SAM

...

premature optimization is

the root of all evil.

...

(by Donald Knuth, in the context of software)

Problems with SAM

1. SAM format was defined **too soon**.
2. The use cases for SAM were **poorly understood**.
3. Dominated by a single institution: **Broad Institute**/
4. Dominated by a single instrument: **Illumina**
5. Too much focus on **performance**.


I AM SAD FOR BAM
NOT BECAUSE WHAT IT IS
BUT BECAUSE WHAT
IT COULD HAVE BEEN
imgflip.com

Silver lining

If you do understand BAM

You look like a

GENIUS!

The SAM format

SAM is a textual based formatting of alignment data

The compressed version of SAM are:

1. **BAM**
2. **CRAM**

SAM, **BAM**, **CRAM** all contain the same information.

We exchange data as **BAM** or **CRAM**

When you read it on the screen --> it is **SAM**

Use the "SAM Specification"

When in doubt consult:

- The **SAM format** specification.

- The **SAM tag** specification.

- The **aligner documentation** to see what **SAM** features it supports.

See the book/course for links.

Making a SAM file

```
# This will be the name to the reference.
REF=refs/ebola.fa

# Make a directory for references.
mkdir -p refs

# Get the reference genome.
efetch -db=nuccore -format=fasta -id=AF086833 > $REF

# Create the reference genome index.
bwa index $REF

# Obtain an ebola short read dataset.
fastq-dump -X 10000 --split-files SRR1972739

# Run the aligner. Make a SAM file.
bwa mem $REF SRR1972739_1.fastq > bwa.sam
```

What is in a SAM file

```
cat bwa.sam | head
```

Prints:

```
@SQ     SN:AF086833.2   LN:18959
@PG     ID:bwa  PN:bwa  VN:0.7.16a-r1181        CL:bwa mem
SRR1972739.1    16      AF086833.2      15684   60      69
SRR1972739.1    2048    AF086833.2      15735   60      33
SRR1972739.2    16      AF086833.2      4919    60      10
SRR1972739.3    0       AF086833.2      2531    60      60
SRR1972739.3    2064    AF086833.2      2618    44      50
SRR1972739.4    4       *       0       0       *       *
...
```

Each line represented one alignment! 11+ columns.

SAM Specification

It is a line oriented format that contains:

---

1. **Header section**,
   Where each line contains metadata

---

2. **Alignment section**
   Where each line describes a single alignment

---

1. SAM Header Section

Headers start with `@`

```
@SQ     SN:AF086833.2 LN:18959
@PG     ID:bwa PN:bwa VN:0.7.16a-r1181  CL:bwa mem refs/eb
```

- Two letter header record codes: `SQ`, `PQ`
- `TAG:VALUE` record values: `SN:AF086833.2`, `PN:bwa`

`SQ` = Sequence
`LN` = Sequence Lenght
`PN` = Program Name

Read it out loud

```
@SQ     SN:AF086833.2 LN:18959
@PG     ID:bwa PN:bwa VN:0.7.16a-r1181
                CL:bwa mem refs/ebola.fa SRR1972739_1.fastq
```

You can read off the header lines like so:

> *The alignment file was created using a reference sequence called* `AF086833.2` *that had the lenght of* `18959` *with a program called* `bwa` *with a version number* `0.7.16a-r1181` *using the command line* `bwa mem refs/ebola.fa SRR1972739_1.fastq`

2. SAM Alignment Section

---

11 *mandatory* columns.

There may be many additional, *optional* columns!

There are substantial differences between short read aligners in what additional columns they produce!

😫 Some *mandatory* columns are useless: `MAPQ`
😫 Many *optional* columns are *very useful*: `MD`

1. `QNAME` - Query name
2. `FLAG` - Alignment flag
3. `RNAME` - Alignment reference name
4. `POS` - Alignment position (left-most coordinate)
5. `MAPQ` - Mapping quality
6. `CIGAR` - Compact Idiosyncrathic Alignment Rep.
7. `RNEXT` - Alignment reference for read mate
8. `PNEXT` - Alignment position of read mate
9. `TLEN` - Template length
10. `SEQ` - Query sequence
11. `QUAL` - Query quality

A short stroll through mandatory SAM fields

Use `samtools` to view SAM data:

```
samtools view bwa.sam | cut -f 1 | head
```

See the samtools help for what it can do:

```
samtools
```

Prints:

```
Program: samtools (Tools for alignments in the SAM format)
Version: 1.5 (using htslib 1.5)

Usage:   samtools <command> [options]
...
```

Column 1: Query Name (QNAME)

The first column indicates the name of the query sequence:

```
samtools view bwa.sam | cut -f 1 | head -4
```

Prints:

```
SRR1972739.1
SRR1972739.1
SRR1972739.2
SRR1972739.3
```

⚠️ Note how `SRR1972739.1` appears twice! It may be the mate or it may be another alignment of the same read. A read may have multiple alignments !!!

Column 2: FLAG

The second column (`FLAG`) is the biggest mistake of the SAM format.

😡 It is an integer number where the binary bits in the integer indicate which statments are true about the alignment.

It boggles the mind and common sense.

⚠️ Everyone will at some point combine flags incorrectly by accident. Leads to untold amounts of wasted time and effort. Always be super vigilant!

Why does it work this way?

They wanted to "save space". You can represent mutually exclusive information as bits. Suppose:

- First bit indicates property `A`
- Second bit indicates property `B`
- Third bit indicates property `C`

```
1 -> in binary format is 001
2 -> in binary format is 010
3 -> in binary format is 011
```

So `3` will mean three things: `A` and `B` but not `C`

What do FLAGS look like?

```
samtools view bwa.sam | cut -f 2 | head -4
```

Flags:

```
16
2048
16
0
```

Checking one flag is not so bad:

```
samtools flags 16
0x10    16      REVERSE
```

Seems ok so far. What is my problem then?

I believe that the *conceptual encoding* of biology into the SAM format was done fundamentally wrong:

- **forward strand** is not the opposite of **reverse**.
- **second in pair** is not the opposite of **first in pair**.

Example: You can only select for or remove the **reverse strand**. You cannot select the **forward strand**.

😈 Removing **reverse** gives you the **forward**...

The challenge of SAM

Is understanding and dealing with the ways flag misrepresent biology and trying to avoid misusing them.

Filtering SAM files.

Filtering SAM files with `FLAGS` is a needed in every analysis:

```
   1   PAIRED          .. paired-end (or multiple-segment) s
   2   PROPER_PAIR     .. each segment properly aligned acco
   4   UNMAP           .. segment unmapped
   8   MUNMAP          .. next segment in the template unmap
  16   REVERSE         .. SEQ is reverse complemented
  32   MREVERSE        .. SEQ of the next segment in the tem
  64   READ1           .. the first segment in the template
 128   READ2           .. the last segment in the template
 256   SECONDARY       .. secondary alignment
 512   QCFAIL          .. not passing quality controls
1024   DUP             .. PCR or optical duplicate
2048   SUPPLEMENTARY   .. supplementary alignment
```

We'll have a lecture on dealing with just these.

Columns 3-4: RNAME and POS

Reference Name (`RNAME`) and Position (`POS`)

```
samtools view bwa.sam | cut -f 3,4 | head -4
```

Prints:

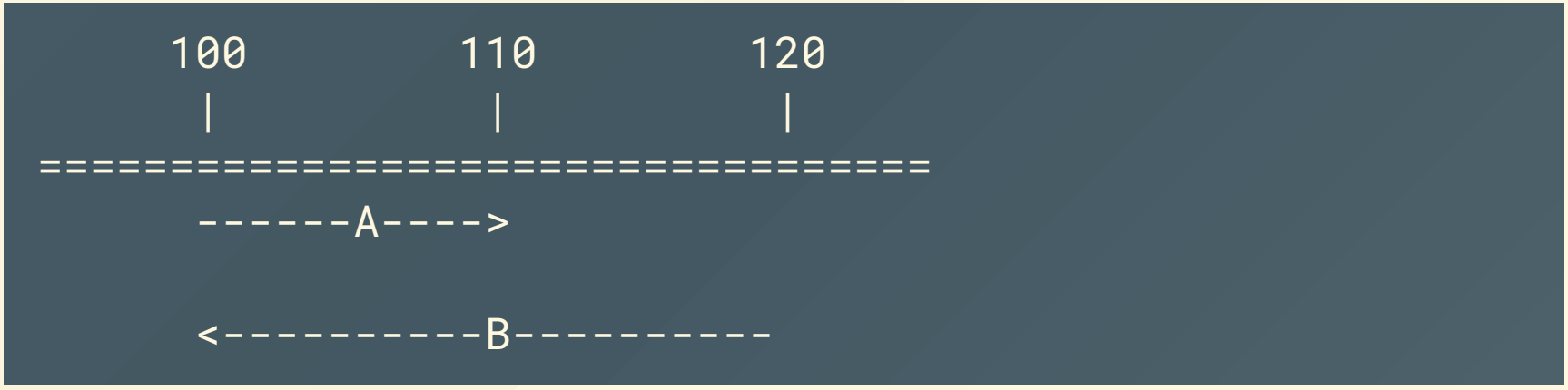
```
AF086833.2      15684
AF086833.2      15735
AF086833.2      4919
AF086833.2      2531
```

The `POS` is the **leftmost** coordinate of the alignment!

The POS column in "practice"

Read `A` aligns on the forward strand.

Read `B` aligns on the reverse strand.



⚠️ The `POS` field for both reads will be `100`

The **SAM** format does not give you the last (end) `110`, `120` coordinates directly. Big oversight there as well.

Column 5-6: (MAPQ) and CIGAR

Mapping Quality (`MAPQ`) and Compact Idiosyncratic Gapped Alignment Representation (`CIGAR`)

```
samtools view bwa.sam | cut -f 5,6 | head -4
```

Prints:

```
60      69M32S
60      33M68H
60      101M
60      60M41S
```

`MAPQ` - is a probability (like FASTQ quality encoding)
`CIGAR` - is our friend from alignment representation.

The truth about MAPQ

Mapping quality is supposed to tell us about the likelihood that the mapping is incorrect.

```
60 --> 1/10^6 (one in a million)
```

⚠️ In reality `MAPQ` are nearly guesses.

Higher numbers are "better" - but do not correspond to real, verifiable probabilities. Weird `MAPQ`s are common. `MAPQ`s are aligner dependent:

- `bwa` sets `MAPQ` to zero for reads that map to more than one location

The truth about CIGAR

⚠️ `CIGAR` strings are almost useless as well. (did you notice a trend here?)

By default `M` will represents `match` or `mismatch` - no really! Think about this for a second.

When you see `100M` you don't actually know how many match and mismatching bases are there.

🤔 Even geniuses can get things ridiculously wrong!

To find out if it is a match or mismatch you have to look at an **optional** field.

Columns 7-9: RNEXT, PNEXT, TLEN

These columns deal with the alignment of the mate in a paired end alignment.

- Read Name of the mate: `RNEXT`
- Postion if the mate: `PNEXT`
- Distance between the leftmost coordinates: `TLEN`

This information is filled in only for paired end alignments.

It is used primarly to investigate fragment positions relative to the reference.

Columns 10-11: SEQ and QUAL

The read sequence (`SEQ`) and sequence quality (`QUAL`)

```
------------------------------
         --------->  A
         <---------  B
```

⚠️ For aligned sequences `SEQ` is always specified on the forward strand!

The SAM reported `SEQ` will be the same for `A` and `B`

The original sequence for `B` is the reverse complement of what is indicated in the `SEQ` column!

Optional Fields

Huge variation exists in what gets filled in.

As you saw before - we can only resolve an alignment if some optionals fields are filled in.

Optional fields don't have an order, you can cut a column anymore!

This makes the SAM format even more complicated than before.

Lots of ad-hoc *cottage software* exists to manage the SAM complexity. Most are bug and error riddled.

Good thing about SAM

I might sound overly harsh in the criticism of SAM.

The good thing about SAM are the invisible parts.

- BZGF compression - block gzip - genius!
- Very fast queries when sorted and indexed!

Good intentions: People who were extremely good at software implementation became decision makers that defined what we should be storing in the file. And that's how SAM was born.

Familiarize yourself with SAM

Keep looking at the files

Decompose fields

Understand what they contains

Some fields of bioinformatics deal only with SAM/BAM files.